



UNIVERSIDAD AUTÓNOMA DEL
ESTADO DE MÉXICO
FACULTAD DE INGENIERÍA



Licenciatura

Ingeniería en computación.

Unidad de aprendizaje:

Proyecto integral de comunicación de datos.

Grupo: 01.

Profesor:

Juan Carlos Escobar González.

Entregable 24: Documentación Final.

Plataforma inteligente de monitoreo de redes.

Equipo de trabajo:

Daniel Dávila Lara.

Jesús Omar Carranza Colín.

Fecha de entrega

28/mayo/2026

1. Resumen Ejecutivo

El proyecto NetGuard se desarrolló con el objetivo de implementar un sistema de monitoreo y detección de anomalías en redes IoT WLAN mediante análisis de tráfico en tiempo real. El sistema integra dispositivos ESP32 comunicándose mediante TCP, UDP y HTTP, un módulo analyzer basado en Scapy para captura y análisis de paquetes, y una API Flask conectada a un dashboard web para visualización y gestión de alertas.

El proyecto logró detectar dispositivos no autorizados mediante análisis de direcciones MAC y establecer un mecanismo de aprobación dinámica desde el dashboard. Se implementó una arquitectura modular compuesta por captura de tráfico, análisis de anomalías y visualización web, operando sobre redes Wi-Fi 2.4 GHz y entornos Windows/Linux. Además, se realizaron pruebas de simulación de tráfico y ataques básicos tipo flood para validar la capacidad de monitoreo y detección del sistema sin afectar la comunicación de los nodos IoT.

2. Alcance del Proyecto

El proyecto cumplió con los objetivos establecidos en el Acta de Constitución

El proyecto contempla el diseño, desarrollo, implementación y evaluación de una plataforma de monitoreo de redes utilizando herramientas de software libre y recursos institucionales. Incluye la recolección de datos, el análisis mediante algoritmos de inteligencia artificial y la generación de reportes. No contempla su implementación comercial ni su despliegue en entornos productivos reales.

Requisitos del Proyecto

- Uso de sistemas Linux para monitoreo.
- Programación en lenguaje Python.
- Disponibilidad de dispositivos IoT o simuladores.
- Herramientas de análisis de red.
- Infraestructura de laboratorio.
- Elaboración de documentación técnica.

Cronograma General

Fase	Actividad	Periodo Estimado
Planeación	Análisis y diseño	Semanas 1–2
Desarrollo	Programación	Semanas 3–7
Integración	Implementación	Semanas 8–9
Validación	Pruebas	Semanas 10–11
Cierre	Documentación	Semana 12

Presupuesto Estimado

Concepto	Costo
Software	\$0.00
Hardware	Institucional
Materiales	Mínimo
Total	Bajo

3. Entregables finales aceptados

Criterios Validados (Resumen):

- **CA-01 — Captura de tráfico.**

Cumple [☒] No cumple [☐]

El sistema registra correctamente el tráfico generado por los dispositivos ESP32 y muestra la información (IP origen, IP destino, protocolo y puerto).

- **CA-02 — Detección de anomalías.**

Cumple [☒] No cumple [☐]

El sistema identifica al menos un evento de tráfico anómalo simulado por un equipo intruso y lo clasifica como alerta.

- **CA-03 — Generación de alertas.**

Cumple [☒] No cumple [☐]

Cuando se detecta una anomalía, el sistema genera una alerta visible en el dashboard y la almacena en la base de datos o en el archivo de logs.

- **CA-04 — Dashboard de visualización.**

Cumple [☒] No cumple [☐]

El dashboard muestra correctamente los dispositivos conectados, el historial de tráfico y las alertas generadas así como gráficas de rendimiento de la red.

4. Exclusiones y Cambios del Alcance

El alcance del proyecto se limitó a la detección básica de anomalías y dispositivos no autorizados dentro de una red IoT WLAN simulada. No se contempló la implementación de mecanismos avanzados de mitigación automática, cifrado de tráfico ni entrenamiento de modelos de inteligencia artificial. Durante el desarrollo, se ajustó el enfoque inicial para priorizar la detección en tiempo real mediante análisis de direcciones MAC y monitoreo de tráfico TCP, UDP y HTTP utilizando dispositivos ESP32 y Scapy.

5. Arquitectura

Tecnologías Utilizadas

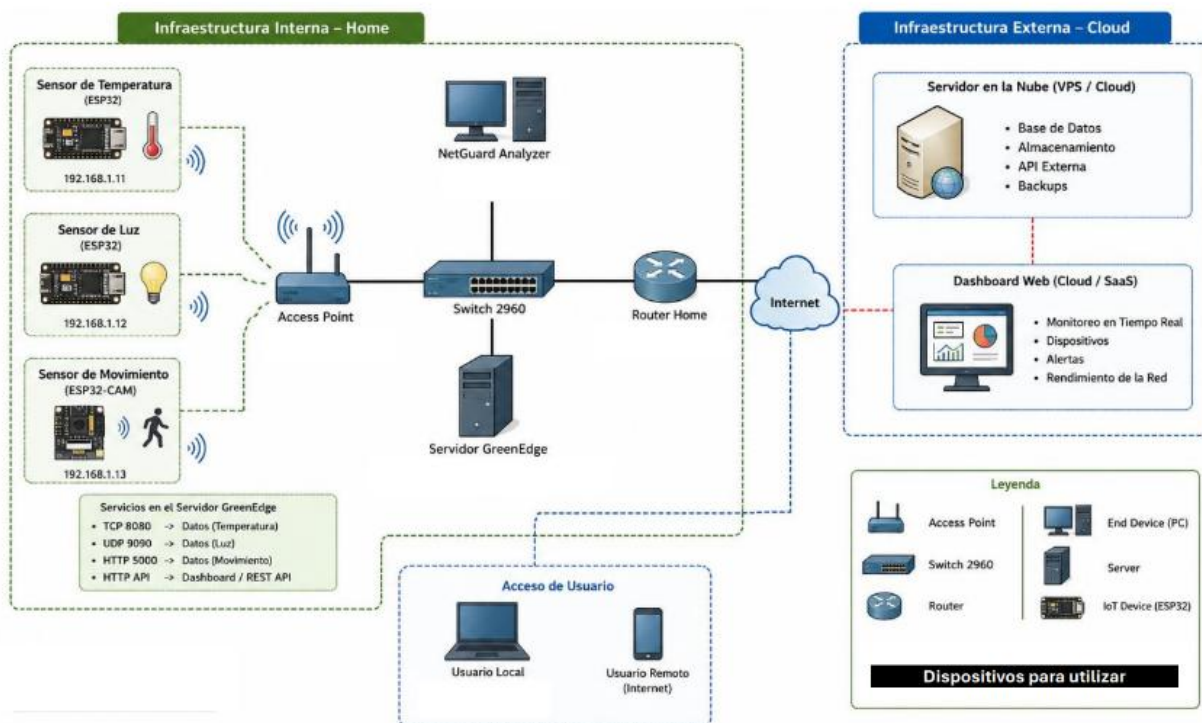
- Sistema Operativo: Windows 11 / Kali Linux 24.04
- Dispositivos IoT: ESP32 DevKit V1
- Captura y Análisis de Tráfico: Python 3.12 + Scapy
- Backend y API REST: Flask
- Comunicación de Red: TCP, UDP y HTTP
- Visualización (Frontend): HTML5, CSS3, JavaScript y Chart.js
- Simulación y Diseño de Red: Cisco Packet Tracer

6. Metodología del Sistema

El proyecto utiliza una metodología basada en monitoreo y análisis de tráfico en tiempo real sobre una red IoT WLAN simulada.

1. Captura de Tráfico: Se implementó un módulo sniffer utilizando Scapy en Python para capturar paquetes TCP, UDP y HTTP generados por dispositivos ESP32 dentro de la red Wi-Fi.
2. Análisis de Anomalías: El analyzer procesa los paquetes capturados para identificar comportamientos anómalos, como detección de dispositivos no autorizados mediante análisis de direcciones MAC y tráfico sospechoso tipo flood.
3. Comunicación y Visualización: Se desarrolló una API REST con Flask para centralizar eventos de red y un dashboard web para monitoreo en tiempo real, visualización de alertas y gestión de dispositivos autorizados.

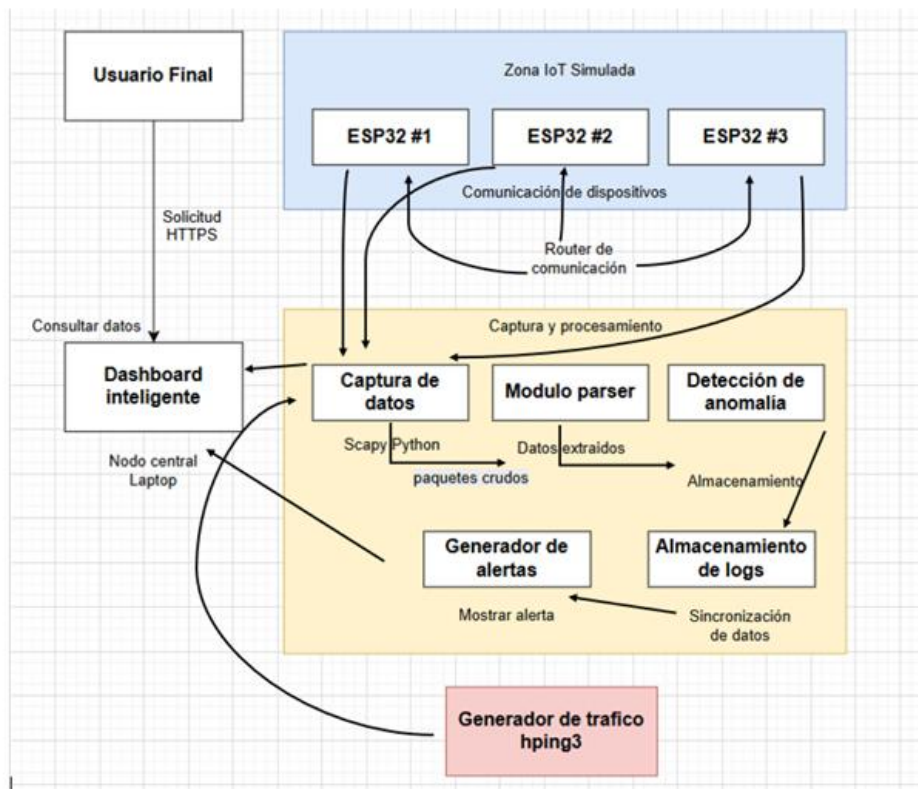
7. Arquitectura Física del Sistema



El sistema utiliza Wi-Fi 2.4 GHz como medio principal debido a la conectividad nativa de los ESP32 y su bajo costo de implementación. La red LAN cableada se emplea en el servidor NetGuard para garantizar estabilidad y menor latencia durante la captura y análisis de tráfico.

Se implementan TCP, UDP y HTTP para simular distintos patrones de comunicación IoT. Scapy permite monitoreo en tiempo real y detección de anomalías a nivel de red. Flask centraliza la comunicación entre analyzer, dispositivos IoT y dashboard web mediante una arquitectura modular y escalable.

8. Arquitectura del Software del Sistema



9. Diseño de registro de Logs

capture.log									
src	netmonitor2	logs	capture.log						
1	10:52:23.733	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60279	Dst: 5000	Bytes: 262	MAX: 262	
2	10:52:29.073	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60280	Dst: 5000	Bytes: 262	MAX: 345	
3	10:52:34.201	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60281	Dst: 5000	Bytes: 262	MAX: 345	
4	10:52:39.509	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60282	Dst: 5000	Bytes: 262	MAX: 345	
5	10:52:44.816	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60283	Dst: 5000	Bytes: 262	MAX: 345	
6	10:52:50.159	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60284	Dst: 5000	Bytes: 262	MAX: 345	
7	10:52:55.455	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60285	Dst: 5000	Bytes: 262	MAX: 345	
8	10:53:00.584	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60286	Dst: 5000	Bytes: 262	MAX: 345	
9	10:53:05.913	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60287	Dst: 5000	Bytes: 262	MAX: 345	
10	10:53:11.233	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60288	Dst: 5000	Bytes: 262	MAX: 345	
11	10:53:16.548	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60289	Dst: 5000	Bytes: 262	MAX: 345	
12	10:53:21.675	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60290	Dst: 5000	Bytes: 262	MAX: 345	
13	10:53:26.805	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60291	Dst: 5000	Bytes: 262	MAX: 345	
14	10:53:32.151	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60292	Dst: 5000	Bytes: 262	MAX: 345	
15	10:53:37.277	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60293	Dst: 5000	Bytes: 262	MAX: 345	
16	10:53:42.570	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60294	Dst: 5000	Bytes: 262	MAX: 345	
17	10:53:47.878	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60295	Dst: 5000	Bytes: 262	MAX: 345	
18	10:53:53.010	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60296	Dst: 5000	Bytes: 262	MAX: 345	
19	10:53:58.173	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60297	Dst: 5000	Bytes: 262	MAX: 345	
20	10:54:03.478	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60298	Dst: 5000	Bytes: 262	MAX: 345	
21	10:54:08.595	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60299	Dst: 5000	Bytes: 262	MAX: 345	
22	10:54:13.733	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60300	Dst: 5000	Bytes: 262	MAX: 345	
23	10:54:19.031	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60301	Dst: 5000	Bytes: 262	MAX: 345	
24	10:54:24.340	HTTP	Org: 10.129.176.13	Dst: 10.129.176.217	Org: 60302	Dst: 5000	Bytes: 262	MAX: 345	

10. Cronograma y Gestión del Tiempo

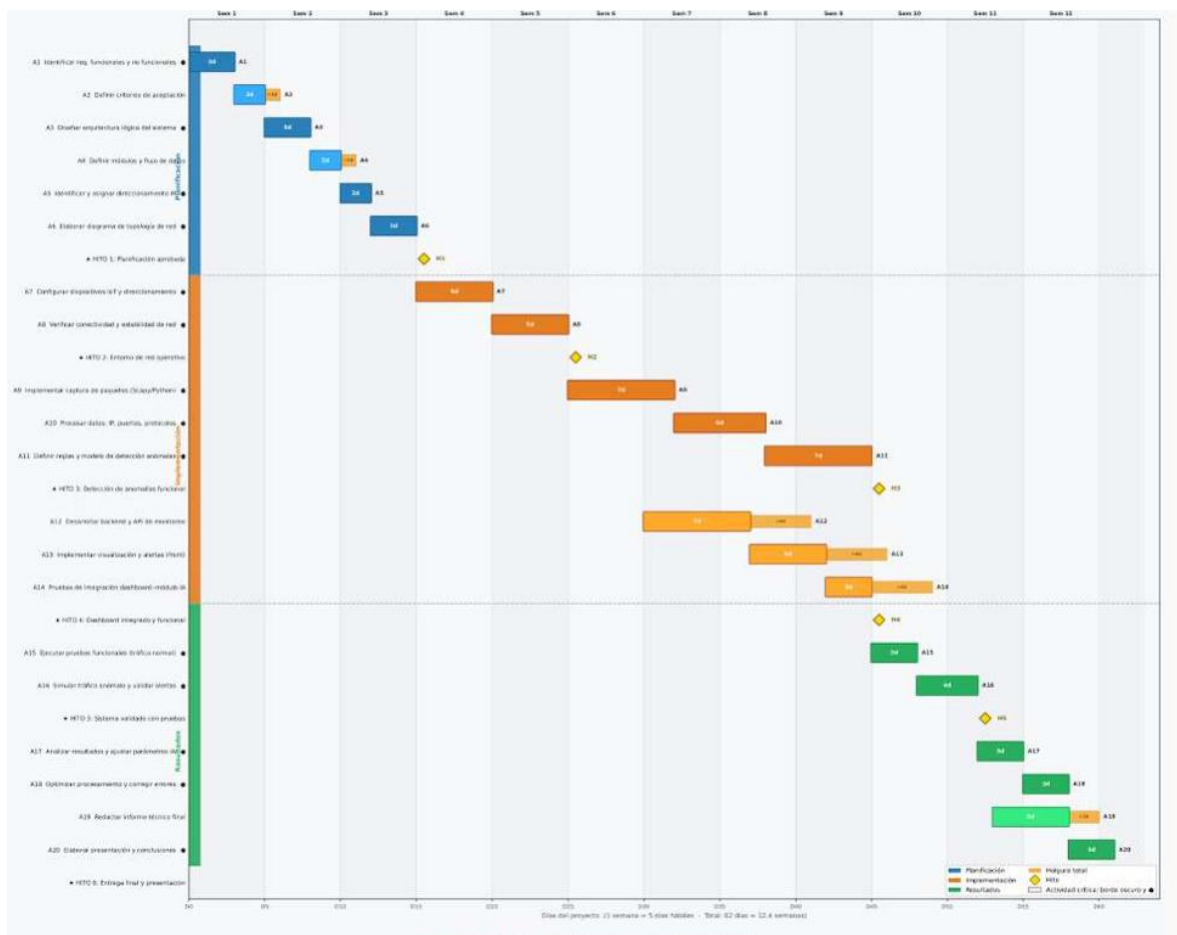
El proyecto tuvo una duración aproximada de 10 semanas.

- Fecha de Inicio Planificada: Marzo 2026.
- Fecha de Fin Planificada: Mayo 2026.
- Fecha de Fin Real: Mayo 2026.

4.2 Desviaciones y Causas

Se registraron ajustes menores durante la fase de integración entre el analyzer y la API Flask.

- Causa Raíz: Problemas de conectividad y captura de paquetes en interfaces WLAN durante las pruebas con Scapy y dispositivos ESP32.
- Acción Tomada: Se realizaron pruebas adicionales de configuración de red, direccionamiento IP estático y depuración de tráfico para estabilizar la comunicación entre módulos sin afectar el cronograma general del proyecto.



11. Gestión de Costos y Finanzas

El proyecto operó bajo un modelo de costos basado principalmente en horas de desarrollo y pruebas realizando ajustes en el presupuesto inicial planeado, esto sin considerarlo como proyecto ya en producción.

Concepto	Presupuesto Base	Costo Real	Variación
Desarrollo e Implementación	\$4,500 MXN	\$4,900 MXN	-\$400 MXN
Dispositivos ESP32 y Red	\$1,200 MXN	\$1,200 MXN	\$0 MXN
Total	\$5,700 MXN	\$6,100 MXN	-\$400 MXN

Uso de la Reserva de Contingencia

Se utilizó parte de la reserva de contingencia para cubrir tiempo adicional durante las pruebas de integración y captura de tráfico en red WLAN.

Justificación: Ajustes de configuración de interfaces de red, depuración de Scapy y pruebas de comunicación entre módulos TCP, UDP y HTTP.

Conclusión Financiera: El proyecto finalizó dentro de un margen aceptable de costos y sin requerir adquisición adicional de hardware o infraestructura especializada.

12. Gestión de Calidad y Pruebas

Caso de Prueba 1 – Captura de tráfico IoT

Campo	Descripción
Nombre	Captura de tráfico de red IoT
Propósito	Validar la captura de tráfico generado por dispositivos IoT simulados.
Precondiciones	Red local configurada, ESP32 conectados, sistema Linux activo y script de captura ejecutándose.
Pasos	1. Encender dispositivos IoT. 2. Generar tráfico de red. 3. Ejecutar captura con Python/Scapy. 4. Verificar recepción de paquetes.
Datos de entrada	Paquetes y tráfico TCP generados por los dispositivos IoT.
Resultado esperado	El sistema captura correctamente los paquetes y los registra.
Resultado obtenido	Captura exitosa de paquetes provenientes de dispositivos IoT simulados.
Incidencias	No se detectaron incidencias.

Caso de Prueba 2 – Detección de anomalías

Campo	Descripción
Nombre	Detección de tráfico anómalo
Propósito	Validar que el sistema detecte comportamientos anómalos en la red.
Precondiciones	Sistema de monitoreo activo y herramienta hping3 disponible.
Pasos	1. Ejecutar tráfico normal. 2. Generar tráfico masivo usando hping3. 3. Observar comportamiento del sistema. 4. Verificar generación de alertas.
Datos de entrada	Incremento artificial de paquetes TCP/ICMP.
Resultado esperado	El sistema detecta el tráfico anómalo y genera una alerta.
Resultado obtenido	Se detectó incremento anormal de tráfico y se registró la anomalía.
Incidencias	Ligero retraso para la visualización del dashboard.

Caso de Prueba 3 – Funcionamiento del dashboard web

Campo	Descripción
Nombre	Validación del dashboard web
Propósito	Verificar que el dashboard muestre métricas y alertas correctamente.
Precondiciones	Backend activo, base de datos funcionando y navegador web disponible.
Pasos	1. Iniciar dashboard web. 2. Generar tráfico en la red. 3. Revisar actualización de métricas. 4. Verificar visualización de alertas.
Datos de entrada	Eventos de red generados por dispositivos IoT.
Resultado esperado	El dashboard muestra gráficas, métricas y alertas correctamente.
Resultado obtenido	Visualización correcta de estadísticas y alertas de red.
Incidencias	Fallo de conectividad con los dos módulos anteriores

13. Gestión de Riesgos e Incidentes

ID	Riesgo	Dueño	Nivel	VME (\$)	Estrategia	Justificación	Acciones Específicas
R-01	Scapy incompatible con versión de Python o sistema operativo	Desarrollador backend	Alto	\$4,800	Evitar	Es mucho más barato prevenir este riesgo desde el inicio que corregirlo durante la implementación. Crear un entorno fijo reduce las probabilidades del riesgo.	1. Crear un entorno virtual Python con versiones fijas de todas las librerías 2. Registrar las versiones exactas en el archivo requirements.txt 3. Probar el entorno en el SO limpio antes de iniciar el desarrollo del módulo de captura.
R-02	Cola de mensajes se satura y se pierden paquetes	Arquitecto de software	Alto	\$3,200	Mitigar	El tiempo dedicado a esta configuración reduce el impacto de reprocesar los datos obtenidos durante la captura y análisis.	1. Implementar un tamaño de procesamiento correcto 2. Agregar un log de paquetes fallidos o dañados para detectar saturación temprana.
R-03	ESP32 con conexión inestable a la red LAN del laboratorio	Ingeniero de redes	Alto	\$2,400	Mitigar	La inestabilidad de los ESP32 es un riesgo que se puede mitigar con configuración adecuada y pruebas previas con el fin de evitar errores durante pruebas funcionales.	1. Configurar correctamente el direccionamiento IP desde Arduino, ya sea dinámico o estático. 2. Ejecutar una prueba de conectividad (ping continuo) antes de cada prueba funcional.
R-04	Incompatibilidad entre hping3 y entorno de red: no se puede simular tráfico de red	Desarrollador backend	Alto	\$3,200	Mitigar	Sin la capacidad de simular tráfico de red no se puede validar el criterio de aceptación de detección de anomalías	1. Realizar una prueba de generación de tráfico con hping3 en la red del laboratorio 2. Documentar los comandos exactos para cada tipo de animalia simulada.
R-05	Infraestructura insuficiente para	Líder de proyecto	Medio	\$4,000	Mitigar	Mitigar con anticipación la disponibilidad y	1. Analizar y considerar correctamente la infraestructura de la red

ID	Riesgo	Dueño	Nivel	VME (\$)	Estrategia	Justificación	Acciones Específicas
	conectar ESP32 a la red LAN					alcance de dispositivos que puede tolerar la red WLAN.	WLAN que se va a utilizar durante el desarrollo del proyecto.
R-06	Perdida de respaldos: pérdida irreversible de código y datos de captura	Líder de proyecto	Medio	\$2,400	Mitigar	Mantener los repositorios remotos actualizados una vez por semana, conservando los cambios o actualizaciones y no perder progreso durante la realización de este proyecto	1. Implementar la Política de Respaldos: Cada cuanto hacer PUSH a el repositorio de GitHub al terminar cada sesión de trabajo. 2. Hacer copia de los archivos en una nube, en este caso documentación o archivos de administración.
R-07	Sistema no ejecutable en equipos distintos al de desarrollo	Líder de proyecto	Medio	\$4,800	Evitar	Este riesgo afecta en que el sistema solo funciona en ciertos equipos, con ciertas características por lo que no cumpliríamos el alcance previsto/	1. Elaborar un manual de instrucciones de instalación paso a paso cubriendo la mayoría de los casos donde se pueda usar nuestro proyecto. 2. Ofrecer alternativas como Docker en caso de que la compatibilidad no sea posible en todos los sistemas.
R-08	API REST de Flask colapsa con errores 500 durante la ejecución del dashboard	Desarrollador Backend	Medio	\$2800	Mitigar	El VME es mediano y el impacto es visible ya que afecta el cumplimiento del alcance, el dashboard es parte fundamental del funcionamiento del proyecto.	1. Realizar las pruebas necesarias y controladas en entornos locales con 5 usuarios simultáneos usando el módulo de dashboard. 2. Manejar una prueba funcional en un entorno de QA antes de presentar la versión final de la herramienta
R-09	Sin documentación del proceso de instalación	Project Manager	Medio	\$800	Mitigar	La acción de mitigación es documentar mientras se desarrolla, lo que tiene costo cero si se hace de forma incremental.	1. Redactar comentarios en cada función al momento de escribirla, no al final del proyecto. 2. Mantener el README.md actualizado con cada cambio.
R-10	Conflicto de IPs entre ESP32 (estáticas) y DHCP del laboratorio	Ingeniero de redes	Medio	\$800	Evitar	Este riesgo es 100% evitable con una configuración correcta de red desde el inicio.	1. Asignar bien las IP's a los ESP32 en un rango definido para una red de clase C. 2. Documentar el esquema de direccionamiento.
R-11	No contar con documentación de pruebas con ESP32:	Lider de proyecto	Medio	\$800	Mitigar	La evidencia de pruebas es obligatoria para	1. Capturar evidencia fotográfica o en video de cada sesión de prueba

ID	Riesgo	Dueño	Nivel	VME (\$)	Estrategia	Justificación	Acciones Específicas
	incumplimiento de alcance					demostrar el cumplimiento del alcance.	con los ESP32 en el laboratorio. 2. Redactar un reporte de pruebas breve (1 página) al finalizar cada sesión de la fase de resultados.
R-12	ESP32 sufre daño físico durante las pruebas	Equipo de hardware	Bajo	\$2,400	Mitigar	Aunque la probabilidad es baja, el impacto puede involucrar irregularidades en el entorno de IoT que se pretende simular	1. Cambiar el ESP32 dañado por uno diferente una vez que ya se descartó que el anterior funcione sin detalles 2. Si solo falla uno, continuar las pruebas con los dispositivos restante y documentarlo como limitación.
R-13	Datos de sesión almacenados sin política de privacidad definida	Líder de proyecto	Bajo	\$480	Mitigar	Aunque la probabilidad es baja en un entorno de laboratorio académico, el impacto legal y ético es alto. La mitigación requiere solo documentar el uso de los datos procesados.	1. Incluir una nota de privacidad en el uso del sistema especificando que el sistema solo procesa datos de tráfico en redes de prueba controladas, no en redes con usuarios reales.
R-14	Código entregado sin documentación de funciones importantes	Project Manager	Bajo	\$400	Mitigar	Es de vital importancia mantener documentadas las versiones del proyecto en caso de la integración de nuevos desarrolladores o equipo de trabajo.	1. Incluir la revisión de documentación con el líder de desarrollo antes de realizar acciones de push en el repositorio remoto del sistema.

14. Lecciones Aprendidas

Lo que funcionó bien

- **Arquitectura Modular:** La separación entre sniffer, analyzer, API Flask y dashboard facilitó el desarrollo, pruebas e integración del sistema.
- **Simulación IoT Realista:** El uso de ESP32 con comunicación TCP, UDP y HTTP permitió generar tráfico variado para pruebas de monitoreo y detección de anomalías.
- **Monitoreo en Tiempo Real:** La integración de Scapy con Flask permitió detectar dispositivos no autorizados y visualizar eventos de red en tiempo real.

Áreas de Mejora (Lo que salió mal)

- **Captura en Redes WLAN:** Se presentaron dificultades iniciales para capturar tráfico inalámbrico correctamente desde interfaces Wi-Fi en Windows.
- **Acción Futura:** Implementar entornos de pruebas dedicados con adaptadores compatibles con modo monitor y sistemas Linux.
- **Gestión de Alertas:** Algunas detecciones iniciales generaron falsos positivos durante pruebas de flood y tráfico normal de red.
- **Acción Futura:** Incorporar reglas de análisis más precisas y mecanismos básicos de correlación de eventos para reducir alertas innecesarias.

Acta de aceptación del proyecto aceptada

Proyecto/Entregable: Netguard. Plataforma inteligente de monitoreo de redes.

Criterios Validados (Resumen):

- **CA-01 — Captura de tráfico.**

Cumple [☒] No cumple [☐]

El sistema registra correctamente el tráfico generado por los dispositivos ESP32 y muestra la información (IP origen, IP destino, protocolo y puerto).

- **CA-02 — Detección de anomalías.**

Cumple [☒] No cumple [☐]

El sistema identifica al menos un evento de tráfico anómalo simulado por un equipo intruso y lo clasifica como alerta.

- **CA-03 — Generación de alertas.**

Cumple [☒] No cumple [☐]

Cuando se detecta una anomalía, el sistema genera una alerta visible en el dashboard y la almacena en la base de datos o en el archivo de logs.

- **CA-04 — Dashboard de visualización.**

Cumple [☒] No cumple [☐]

El dashboard muestra correctamente los dispositivos conectados, el historial de tráfico y las alertas generadas así como gráficas de rendimiento de la red.

Observaciones del usuario/cliente:

Decisión:

☒ Aceptado: El entregable cumple con todos los criterios de aceptación definidos.

☐ Aceptado con cambios: El entregable es funcional, pero requiere los ajustes menores listados en el "Registro de Hallazgos" (ID: _____).

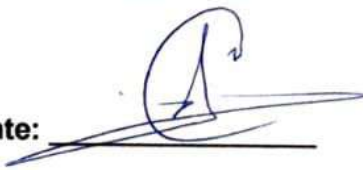
☐ Rechazado: El entregable no cumple con los criterios críticos

(_____)

Se requiere una solicitud de cambio formal.

Compromisos/Fechas:

Firma usuario cliente:

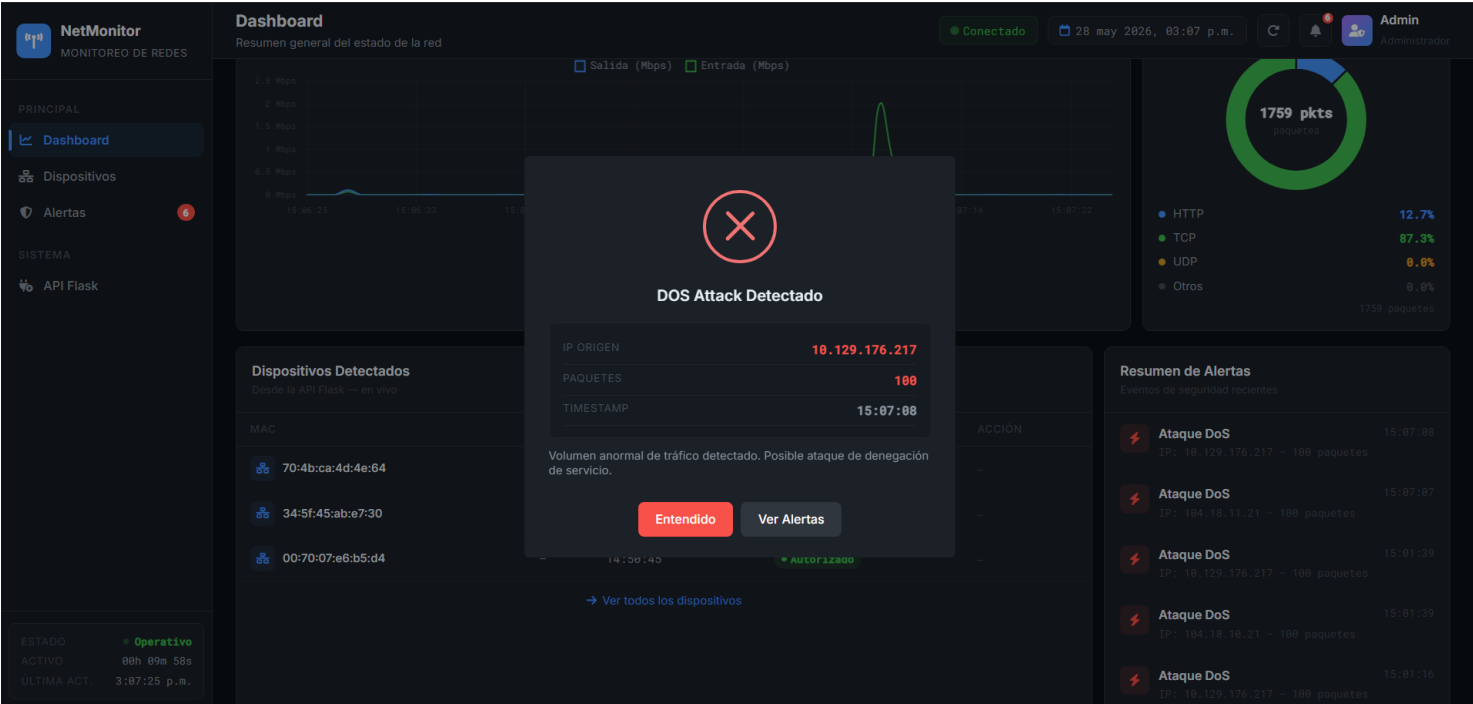


Firma director del proyecto:



Fecha: 28 Mayo de 2026

Anexos de imágenes



NetMonitor

MONITOREO DE REDES

PRINCIPAL

Dashboard

Dispositivos

Alertas

SISTEMA

API Flask

ESTADO

Operativo

ACTIVO

00h 00m 27s

ULTIMA ACT.

5:04:33 p.m.

Dashboard

Resumen general del estado de la red

DISPOSITIVOS AUTORIZADOS

0

+4 pendientes

TRÁFICO TOTAL

0.3 GB

↑Tiempo real

ANCHO DE BANDA

0.0 Mbps

↑Tiempo real

ALERTAS ACTIVAS

0

↑Tiempo real

Tráfico de Red

Mbps en tiempo real — entrada y salida

En vivo

Distribución de Tráfico

Por protocolo — enviado por analyzer

paquetes

HTTP

0.0%

TCP

0.0%

UDP

0.0%

Otros

0.0%

Nuevo dispositivo detectado

?

IP

10.129.176.12

MAC

70:4b:ca:4d:4e:64

TIMESTAMP

17:04:30

ESTADO

Pendiente

¿Deseas autorizar este dispositivo en la red?

Aceptar

Ignorar

Dispositivos Detectados

Desde la API Flask — en vivo

MAC	IP	TIMESTAMP	ESTADO	ACCIÓN
70:4b:ca:4d:4e:64	10.129.176.12	17:04:30	Pendiente	Aprobar

Resumen de Alertas

Eventos de seguridad recientes

Sin alertas recientes

Ver todas las alertas

NetMonitor

MONITOREO DE REDES

PRINCIPAL

Dashboard

Dispositivos

Alertas

SISTEMA

API Flask

ESTADO

Operativo

ACTIVO

00h 00m 46s

ULTIMA ACT.

5:04:52 p.m.

Dashboard

Resumen general del estado de la red

DISPOSITIVOS AUTORIZADOS

0

+4 pendientes

TRÁFICO TOTAL

0.3 GB

↑Tiempo real

ANCHO DE BANDA

0.0 Mbps

↑Tiempo real

ALERTAS ACTIVAS

0

↑Tiempo real

Tráfico de Red

Mbps en tiempo real — entrada y salida

En vivo

Distribución de Tráfico

Por protocolo — enviado por analyzer

313 pkts

paquetes

HTTP

46.0%

TCP

0.0%

UDP

54.0%

Otros

0.0%

313 paquetes

Dispositivo autorizado

✓

MAC 70:4b:ca:4d:4e:64 agregada correctamente.

Dispositivos Detectados

Desde la API Flask — en vivo

MAC	IP	TIMESTAMP	ESTADO	ACCIÓN
70:4b:ca:4d:4e:64	10.129.176.12	17:04:30	Pendiente	Aprobar

Resumen de Alertas

Eventos de seguridad recientes

Sin alertas recientes

Ver todas las alertas

NetMonitor

MONITOREO DE REDES

PRINCIPAL

Dashboard

Dispositivos

Alertas

SISTEMA

API Flask

ESTADO

ACTIVO

ULTIMA ACT.

Operativo

00h 01m 23s

5:05:29 p.m.

Dashboard

Resumen general del estado de la red

Conectado

28 may 2026, 05:05 p.m.

Admin

Administrador

DISPOSITIVOS AUTORIZADOS

1

+4 pendientes

TRÁFICO TOTAL

0.3 GB

↑Tiempo real

ANCHO DE BANDA

0.0 Mbps

↑Tiempo real

ALERTAS ACTIVAS

0

↑Tiempo real

Tráfico de Red

Mapa en tiempo real — entrada y salida

En vivo

Dispositivo ignorado

MAC 80:30:49:a1:f2:43 marcada como ignorada.

Distribución de Tráfico

Por protocolo — enviado por analyzer

326 pkts

paquetes

HTTP 46.0%

TCP 0.0%

UDP 54.0%

Otros 0.0%

326 paquetes

Dispositivos Detectados

Desde la API Flask — en vivo

MAC	IP	TIMESTAMP	ESTADO	ACCIÓN
80:30:49:a1:f2:43	10.129.176.207	17:05:18	Pendiente	Aprobar

Resumen de Alertas

Eventos de seguridad recientes

Sin alertas recientes

NetMonitor

MONITOREO DE REDES

PRINCIPAL

Dashboard

Dispositivos

Alertas

SISTEMA

API Flask

ESTADO

ACTIVO

ULTIMA ACT.

Operativo

00h 03m 49s

5:07:55 p.m.

Dashboard

Resumen general del estado de la red

Conectado

28 may 2026, 05:07 p.m.

Admin

Administrador

DISPOSITIVOS AUTORIZADOS

1

+3 pendientes

TRÁFICO TOTAL

0.3 GB

↑Tiempo real

ANCHO DE BANDA

0.1 Mbps

↑Tiempo real

ALERTAS ACTIVAS

5

↑Tiempo real

Tráfico de Red

Mapa en tiempo real — entrada y salida

En vivo

SYN Flood Detectado

IP ORIGEN 10.129.176.207

SYN PKTS 50

TIMESTAMP 17:07:43

Inundación de paquetes SYN. Posible saturación de conexiones TCP.

Entendido

Ver Alertas

Distribución de Tráfico

Por protocolo — enviado por analyzer

382 pkts

paquetes

HTTP 46.3%

TCP 0.0%

UDP 53.7%

Otros 0.0%

382 paquetes

Dispositivos Detectados

Desde la API Flask — en vivo

MAC	IP	TIMESTAMP	ESTADO	ACCIÓN
80:30:49:a1:f2:43	10.129.176.207	17:05:18	Ignorado	-

Resumen de Alertas

Eventos de seguridad recientes

SYN Flood

17:10.129.176.207 — 50 SYN pkts

17:07:43

NetMonitor

MONITOREO DE REDES

PRINCIPAL

Dashboard

Dispositivos

Alertas

SISTEMA

API Flask

ESTADO

Operativo

ACTIVO

00h 01m 39s

ULTIMA ACT.

5:05:45 p.m.

Dispositivos

Gestión y monitoreo de dispositivos conectados

Conectado

28 may 2026, 05:05 p.m.

Admin

Administrador

Buscar MAC, IP...

Todos

Autorizados

Pendientes

Ignorados

Todos los Dispositivos

Conectado

Detectados por el analizador de red

MAC	IP	TIMESTAMP	ESTADO	ACCIÓN
80:30:49:a1:f2:43	10.129.176.207	17:05:18	Ignorado	-
70:4b:ca:4d:4e:64	10.129.176.12	17:04:30	Autorizado	-
34:5f:45:ab:e7:30	10.129.176.13	17:04:30	Pendiente	Aprobar
ae:98:69:20:3f:7c	104.16.185.241	17:04:29	Pendiente	Aprobar
4c:03:4f:0e:97:96	10.129.176.217	17:04:29	Pendiente	Aprobar

```
[SYN FLOOD DETECTED]
{
  "event": "SYN_FLOOD",
  "timestamp": "17:00:53",
  "ip_src": "10.129.176.207",
  "syn_packets": 1404
}
[SERVER] 200

[DOS DETECTED]
{
  "event": "DOS_ATTACK",
  "timestamp": "17:00:53",
  "ip_src": "10.129.176.217",
  "packets": 1395
}
[SERVER] 200
[PROTO STATS] HTTP=46.56% TCP=0.0% UDP=53.44% Otros=0.0% Total=247 → 200

[DOS DETECTED]
{
  "event": "DOS_ATTACK",
  "timestamp": "17:01:02",
  "ip_src": "10.129.176.207",
  "packets": 425
}
[PROTO STATS] HTTP=46.59% TCP=0.0% UDP=53.41% Otros=0.0% Total=249 → 200
[SERVER] 200
[PROTO STATS] HTTP=46.61% TCP=0.0% UDP=53.39% Otros=0.0% Total=251 → 200

[SYN FLOOD DETECTED]
{
  "event": "SYN_FLOOD",
  "timestamp": "17:01:08",
  "ip_src": "10.129.176.207",
  "syn_packets": 2815
}
[SERVER] 200

[DOS DETECTED]
{
  "event": "DOS_ATTACK",
  "timestamp": "17:01:08",
  "ip_src": "10.129.176.217",
  "packets": 2819
}
[SERVER] 200
```